Nguyễn Phạm Trí CE192008

REPORT HOMESTAY BOOKING MANAGEMENT SYSTEM

1. Introduction

This project develops a **Homestay Booking Management System** using Java, applying Object-Oriented Programming (OOP) principles and Computational Thinking methodology.

The system allows users to:

- Manage Tours
- Manage Bookings
- Manage Homestays
- Validate scheduling conflicts
- Save data
- Generate statistics and reports

The architecture follows a layered design separating:

- Model layer
- Controller layer
- File handling layer
- Utility layer
- View layer

This separation improves maintainability, scalability, and clarity.

---

2. Problem Analysis

The system must handle:

1. Tour creation with schedule validation
2. Booking creation with date constraints
3. Homestay capacity validation
4. Overlap detection between tours
5. Data persistence using text files
6. searching, updating and displaying data
7. Remove a Booking by bookingID
8. Update a Booking by bookingID

9. List all Booking by the fullName or a partial fullName
10. Statistics on the total number of tourists who have booked homestays
11. List the Tours with departure dates earlier than the current date
12. List the total Booking amount for tours with departure dates later than the current date

Main constraints:

- Tour ID and Booking ID must be unique
- Booking date must be before departure date
- Departure date must be before end date
- Number of tourists ≤ homestay capacity
- Tours in same homestay cannot overlap
- Phone number must match VietNam prefixes

---

3. Computational Thinking

3.1 Decomposition

The problem was divided into smaller components:

1. Data Model
    - Booking
    - Tour
    - Homestay
2. Business Logic
    - Controllers
3. File Handling
    - FileHelper classes
    - IFileReadWrite interface
4. Validation
    - Validator utility
5. User Interaction
    - View class

Each responsibility is isolated.

3.2 Pattern Recognition

Common patterns identified:

- CRUD operations for Tour and Booking
- Input validation pattern
- File read/write pattern
- ID uniqueness check
- Date comparison logic
- Overlapping interval detection

Recurring logic was extracted into utility classes (Validator, FileUtils).

3.3 Abstraction

Abstraction is applied through:

- Interface IFileReadWrite<T>
- Controllers hiding list manipulation
- View hiding input/output logic
- FileUtils abstracting file operations

Example:

BookingFileHelper implements IFileReadWrite<Booking>

The system does not depend on file type details at upper layers.

3.4 Algorithm Design

The system applies several core algorithms, including:

- Schedule overlap detection
- CRUD operations
- Date validation
- Searching and filtering
- Statistical aggregation
- File persistence logic
- State synchronization logic

3.4.1 Tour Schedule Overlap Detection Algorithm

Ensure that two tours in the same homestay do not overlap in time.

Given two time intervals:

Existing tour:
[a1, a2]

New tour:
[dep, end]

Two intervals overlap if:

NOT (end < a1 OR dep > a2)

3.4.2 Add New Tour Algorithm

Add a tour while ensuring data integrity.

3.4.3 Add Booking Algorithm

Add a booking while maintaining consistency.

Constraint Handling:

Booking date must be before departure date

Phone must match VietNam

Booking ID must be unique

3.4.4 Remove Booking Algorithm

Remove booking and update tour booking status.

3.4.5 Update Tour Algorithm

Update tour while preserving constraints.

Critical Validation:

Overlap must be rechecked after:

HomeID change

Date change

3.4.6 Search Booking by Name Algorithm

Return bookings matching partial name.

3.4.7 Statistics: Total Tourists per Homestay

Calculate total tourists booked for each homestay.

3.4.8 File Read Algorithm

Load data from UTF-16LE text file.

3.4.9 File Write Algorithm

Persist data.

3.4.10 Dirty Flag Algorithm

Avoid unnecessary file writes.

---

4. OOP Design (UML)

Layered Architecture

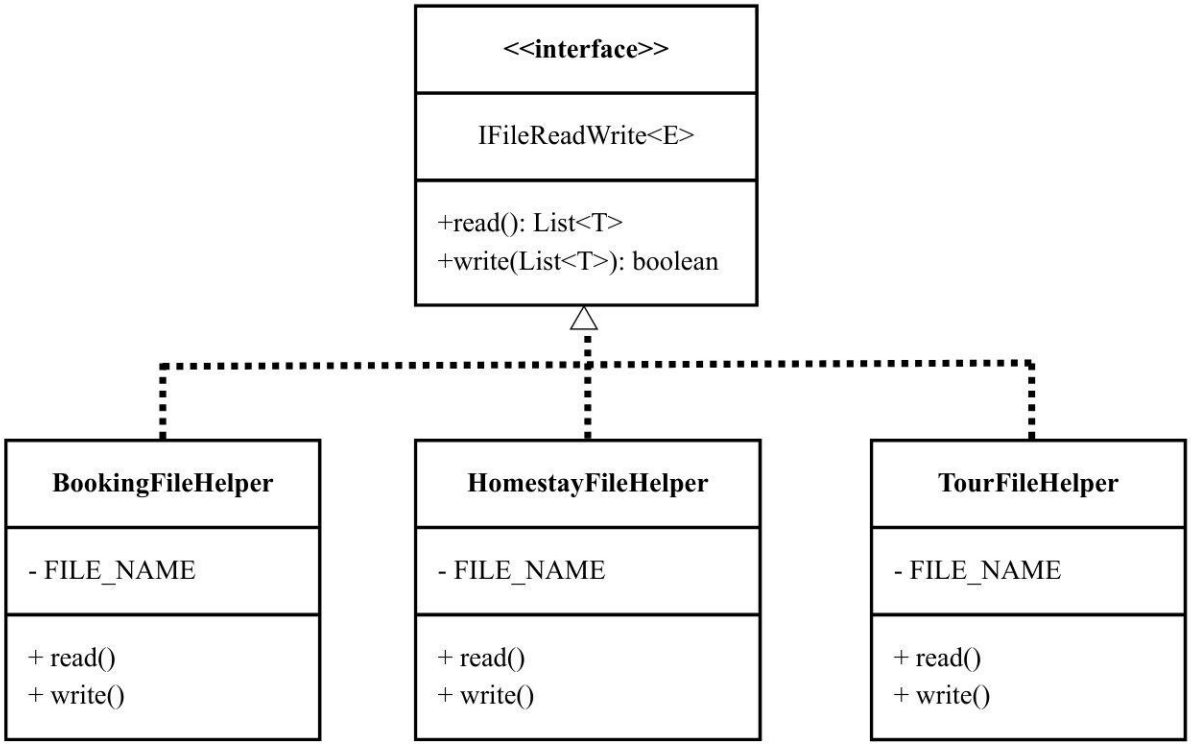
Main
→ Controllers
→ Models
→ FileHelpers
→ FileUtils / Validator
→ View

Main Class Responsibilities:

- Display menu
- Call corresponding controller
- Handle exit/save logic

Class Diagram

fileio layer



Model layer

Booking

- bookingID : String
- fullName : String
- tourID : String
- bookingDate : Date
- phone : String

+ getters/setters

Tour

- tourID : String
- tourName : String
- time : String
- price : int
- homeID : String
-departureDate:Date
- endDate : Date
- numberTourist:int
- booking : boolean

+ getters/setters
+ isScheduleOverlapped()

Homestay

- homeID : String
- homeName : String
- roomNumber : int
- address : String
- maxCapacity : int

+ getters/setters

controller layer

BookingController

- bookings:List<Booking>
– fileHelper: IFileReadWrite<Booking>

+ load()
+ save()
+ add()
+ existsBookingID()
+ findById()
+hasBookingForTour()
+getAll()
+remove()

TourController

– tours: List<Tour>
– fileHelper: IFileReadWrite<Tour>

+ load()
+ save()
+ add()
+ existsBookingID()
+ findById()
+hasBookingForTour()
+getAll()
+isScheduleOverlapped()

HomestayController

– homestays: List<Homestay>
– fileHelper: IFileReadWrite<Homestay>

+ load()
+findById()
+getAll()

utils layer

FileUtils

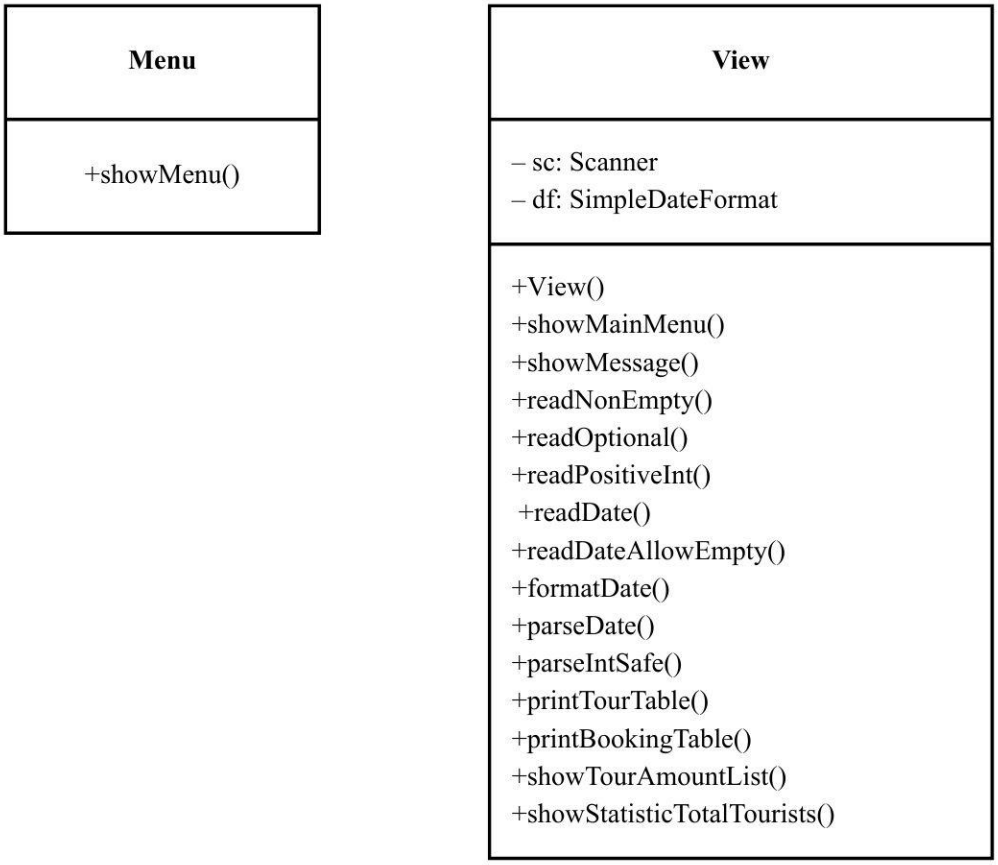
+ readAllLines()
+ writeAllLines()

Validator

– PHONE_PATTERN: String
– VIETTEL_PREFIXES: Set<String>
– VNPT_PREFIXES: Set<String>

+validHomeID()
+validTourID()
+validBookingID()
+validPhone()
+isViettelOrVNPT()

view layer


Menu
+showMenu()
View
– sc: Scanner
– df: SimpleDateFormat
+View()
+showMainMenu()
+showMessage()
+readNonEmpty()
+readOptional()
+readPositiveInt()
+readDate()
+readDateAllowEmpty()
+formatDate()
+parseDate()
+parseIntSafe()
+printTourTable()
+printBookingTable()
+showTourAmountList()
+showStatisticTotalTourists()

Main
- BCtrl : BookingController - HCtrl : HomestayController - TCtrl : TourController - view : View - df : SimpleDateFormat - dirty : boolean
+ main(String[]) - addNewTour() - updateTourById() - listToursDepartureEarlierThanToday() - listTotalBookingAmountForToursLaterThanToday() - addNewBooking() - removeBookingById() - updateBookingById() - listBookingsByName() - statisticTotalTouristsBookedEachHomestay() - exit()

5. Flowchart

PSEUDOCODE

MAIN PROGRAM

```
BEGIN PROGRAM

CREATE BookingController
CREATE TourController
CREATE HomestayController
CREATE View

CALL load() for all controllers

WHILE TRUE
   DISPLAY menu
   READ choice

   SWITCH choice
      CASE 1: addNewTour()
      CASE 2: updateTour()
      CASE 3: listToursDepartureEarlierThanToday()
      CASE 4: listTotalBookingAmountForToursLaterThanToday()
      CASE 5: addNewBooking()
      CASE 6: removeBookingById()
      CASE 7: updateBookingById()
      CASE 8: listBookingsByName()
      CASE 9: statisticTotalTouristsBookedEachHomestay()
      CASE 10: exit()
END WHILE
```

ADD NEW TOUR

```
REPEAT
   INPUT tourID
   IF invalid OR duplicate THEN retry

   INPUT tourName
   INPUT time
   INPUT price

   INPUT homeID
   VALIDATE homeID exists

   INPUT departureDate
   INPUT endDate
   IF endDate < departureDate THEN retry

   INPUT numberTourist
   IF numberTourist > maxCapacity THEN retry

   IF schedule overlapped THEN abort

   ADD new Tour to list
   SET dirty = true

   ASK continue?
UNTIL user stops
```

UPDATE TOUR

```
INPUT tourID
FIND tour
IF not found THEN return

OPTIONAL update each field

IF overlap THEN abort

APPLY changes
SET dirty = true
```

ADD BOOKING

```
INPUT bookingID
VALIDATE unique

INPUT fullName

INPUT tourID
CHECK exists

INPUT bookingDate
CHECK bookingDate < departureDate

INPUT phone
VALIDATE prefix

ADD booking
SET tour.booking = true
SET dirty = true
```

REMOVE BOOKING

```
INPUT bookingID
FIND booking
REMOVE booking
SET dirty = true

IF no more booking for that tour
   SET tour.booking = false
```

UPDATE BOOKING

```
INPUT bookingID
FIND booking

OPTIONAL update fields

VALIDATE bookingDate < departureDate

UPDATE booking

SYNC booking flags
SET dirty = true
```

LIST PAST TOURS

START

For each Tour

If Departure Date < Today

Display Tour

END

LIST FUTURE TOUR TOTAL AMOUNT

START

For each Tour

If Departure Date > Today

Add to list

Sort list by Total Amount descending

Display list

END

SEARCH BOOKING BY NAME

START

Input search keyword

For each Booking

If name contains keyword

Display booking

If no result

Show "Not found"

END

STATISTICS – TOTAL TOURISTS EACH HOMESTAY

START

```
For each Homestay
    Set total = 0

For each Tour
    If booking = TRUE
        Add numberTourist to that homestay

Display result
```

END

EXIT PROGRAM

START Exit

```
If dirty = FALSE
    Exit program

Ask user: Save changes?
If Yes
    Save Tour file
    Save Booking file

Exit program
```

END

(Overlap)

```
For each Tour in same Homestay
    If new schedule overlaps existing schedule
        Return TRUE

Return FALSE
```

6. Conclusion

The project successfully demonstrates:

- Object-Oriented Design
- Layered Architecture
- Data validation techniques
- File persistence handling
- Overlap detection algorithm
- Computational Thinking methodology

The modular structure allows easy extension.